

AI モデルフォールバックフロー

supabase/functions/inbound-email/, match-batch/, match-score/, auto-match/, poll-email/ の実装に基づく。

歴史的注意 (2026-05-19 / コミット 139a4f2 で AI 廃止 + a4dc3b4 でデッドコード全削除)

inbound-email の AI 解析パス (STEP1 関連性チェック + STEP5 人材情報抽出) は完全に廃止された。classifyInboundRelevance / generateJSONSmart / generateJSONWithCerebras / generateJSONWithGroq / generateJSON (kind='candidate'/'project') / buildCandidateGroqPrompt / buildProjectGroqPrompt もすべて削除済み (Grep ヒット 0)。メール解析は AI を一切呼ばず、regex + 文章スキャン + skill_master DB 照合のみで構造化抽出する。

マッチング再設計 (2026-05-22 / コミット b35df40) 新 Edge Function match-batch を導入。ルールベース事前フィルタ + バッチ AI 採点で 1 案件 = 1 AI コールに圧縮。auto-match も match-batch を内部呼び出しするように書き直され、Gemini 単発から Cerebras → Groq → Gemini フォールバック付きに昇格した。

現行 AI 使用箇所マップ

Edge Function / 場所	用途	AI 使用	フォールバック順
inbound-email メール解析	候補者・案件の構造化抽出	不使用	—
inbound-email 自動マッチ (AUTO_MATCH_ENABLED=true 時)	即時スコア計算	使用	Gemini 単発 (matchCandidateToProject)
match-batch Edge Function (UI 高速/全件・auto-match 内部呼び出し)	バッチスコア計算 (ルール事前フィルタ + topN を 1 コール採点)	使用	Cerebras llama3.1-8b → Groq llama-3.3-70b-versatile → Gemini gemini-2.5-flash。3 段すべて失敗時は ルールスコアで全代替 (usedModel='rule')
match-score Edge Function (UI 単発・duplicate 検出付き)	1 ペアのスコア計算	使用	同じ 3 段フォールバック。失敗時はエラー (UI でリトライ可能)
auto-match Edge Function (毎朝 JST 9:00 cron)	バッチスコア計算	使用	match-batch を内部呼び出し → 同じ 3 段フォールバック
poll-email メール種別バッチ分類	同一受信箱内の candidate/project/other 判定	使用 (既定 OFF)	Gemini gemini-2.5-flash-lite 単発 (バッチサイズ最大 20)
ブラウザ (人材・案件登録 UI)	テキスト解析	不使用 (Phase 4.11 で「AI で登録」廃止、登録	—

Edge Function / 場所	用途	AI 使用	フォールバック順
		ボタンは inbound- email regex 経 路に一本 化)	

メール解析の現行パイプライン（AI 不使用）

inbound-email/index.ts の STEP5 は AI を使わず以下のステップで動く。

```
Outlook 受信メール
├─ poll-email (5 分ごと pg_cron・最大 50 件/アカウント)
│   └─ inbound-email
│       ├── [STEP0-2] メタ情報・本文・添付の受け取りと検証
│       ├── [STEP2.5] 研修報告 / 案件紹介スキップ
│       │   ├── TRAINING_REPORT キーワード（「研修内容について報告します」「【本
│       │   │   日の作業進捗】」等）
│       │   ├── PROJECT_SOLICITATION キーワード（「案件情報のご紹介ございま
│       │   │   す」「要員様のご提案をお願いいたします」等）
│       │   └── 該当時は HTTP 200 + skipped で即返す（人材メールボックスへの誤
│       │       投函対策）
│       ├── [STEP3] Word/Excel 添付をテキスト変換（PDF は Storage 保存のみで解
│       │   析せず）
│       ├── [STEP4] Google Drive / Sheets / Docs URL を検出して取得
│       ├── [STEP5] 構造化抽出（AI 不使用）
│       │   ├── decodeHtmlEntities（&等を実体化）
│       │   ├── stripUrlsForSkillMatching（URL を空白置換 → PHP/HTTPS の誤
│       │       マッチ防止）
│       │   ├── stripSenderSignature（送信者署名以降を除去）
│       │   └── extractAndRemoveSkills（skill_master DB 照合・スペースなし
│       │       比較対応）
│       │   ├── 本文：厳密照合（資格は certContext 内のみ）
│       │   └── 添付：フォーマット崩れ対応（資格は looseCert=true で全文
│       │       fallback）
│       ├── filterBySkillRating（スキルシート A~E 評価のうち D/E を除
│       │   外）
│       ├── extractCandidateFieldsRegex + flexLabel
│       │   ├── 氏名・最寄駅・都道府県・経験年数・希望単価・参画時期・希望案
│       │   │   件
│       │   └── 【単 価】等の全角スペース入りラベル・◆氏名◆等のデコレータに
│       │       対応
│       ├── SEP に 】 を含めて 【単価】65万 のような囲み記号にも対応
│       └── inferPrefectureFromStation（駅 → 都道府県マップで署名由来の
│           誤判定を上書き）
│           ├── 約 254 駅・32 都道府県をカバー
│           └── 未収載駅は console.log('[station_unmapped]',
│               station) で記録
│           ├── extractFromProse（PROSE_ROLES / PROSE_INDUSTRIES）
│           │   ├── isPhaseTableHeader でフェーズ表ヘッダー行を除外
│           └── splitMultiCandidateBody（1 メール = 複数候補者を分割・区切り
```

```
線 2 本以上で発動)
    |      └─ extractAgentComment (エージェント所感を最大 500 字で抽出)
    └─ [STEP6] 重複判定 (名前完全一致 + スキル Jaccard  $\geq 0.4$  →
duplicate_flag=true・駅違いは別人扱い)
    └─ [STEP7] DB 保存 (candidates / candidate_skills / ai_logs。
ai_logs.model='no-ai')
    └─ [STEP8] AUTO_MATCH_ENABLED=true なら matchCandidateToProject
経由で即時スコア (任意・既定 OFF)
```

案件メールも同じ前処理 + `extractFieldTwoPhase` で「場所・単価・時期・備考・募集人数・契約形態・クライアント・商流・精算幅・面談形式」を抽出する (コミット `c8be840` で人材経路に統一)。

ポイント:

- `inbound-email` 用に必要な Secrets は `SUPABASE_URL / SUPABASE_SERVICE_ROLE_KEY` のみ。
`GEMINI_API_KEY` 等は不要 (即時マッチングを使う場合のみ追加)
- 案件メールの処理は `app_config.inbound_project_enabled='true'` のときだけ実行される (既定 OFF)
- 手動登録ボタンは `force=true` で `DEDUP / SENDER_DAILY_LIMIT / inbound_project_enabled` をバイパス

match-batch (新方式・1 案件 = 1 AI コール)

マッチング AI 使用量削減のため、コミット `b35df40` で導入。ルールベース事前フィルタ (100pt) で全候補者を採点し、上位 topN (既定 10 名) だけ AI に再採点させる。

ルールベーススコア `calcRuleScore` (0~100pt)

観点	配点	ロジック
スキル一致	最大 40pt	<code>required_skills</code> がある場合のみ算出。完全一致 1pt / includes 部分一致 0.5pt → $(\text{hits}/\text{required.length}) * 40$ 。required 空のときは固定 +20
経験年数	最大 15pt	10年=15 / 7年=12 / 5年=8 / 3年=4 / 1年=2
単価	最大 15pt	<code>budgetMax==null</code> なら +15 固定、範囲内 +15、上限+10% +8、上限+20% +3
勤務地	最大 20pt	同じ都道府県 (接尾辞除去 includes 一致) +20、フルリモート (<code>/フルリモート 完全リモート 100[%]リモート/</code>) +20、居住地不明 +5
リモート	最大 10pt	<code>!isFullRemote && remoteAvailable && /リモート remote 在宅/i.test(remotePolicy)</code> で +10

AI 再採点

- 1 コールで topN 名 (既定 10) を一括採点
- 各候補者に `ruleScore` を埋め込み、**「ruleScore を参考にしつつ役割・経歴・希望職種で再採点」**を AI に指示
- `summary` は **150 字以内** で「必須スキル合致 → 経験年数 → 単価 → 勤務地リモート → 懸念点」の優先順

- 出力形式: [{"id": "...", "score": 整数, "summary": "150字以内"}, ...]

フォールバック

```

Cerebras llama3.1-8b (20s タイムアウト)
├─ 成功 → スコアを返す
└─ 失敗
    └─ Groq llama-3.3-70b-versatile (25s)
        ├─ 成功 → スコアを返す
        └─ 失敗 (429 TPD 超過 / タイムアウト等)
            └─ Gemini gemini-2.5-flash (30s)
                ├─ 成功 → スコアを返す
                └─ 失敗
                    └─ ルールスコアで全代替 (usedModel='rule', summary
空)

```

戻り値構造:

- results**: topN 件 (AI スコア + summary)
- ruleOnly**: 残り (ルールスコアのみ・summary 空)
- usedModel**: 'rule' | CEREBRAS_MODEL | GROQ_MODEL | GEMINI_MODEL

match-score (UI 単発・duplicate 検出付き)

1 ペアの詳細スコアと「重複疑い」フラグを返す軽量経路。MatchingPage の詳細パネルや CandidatePage の手動チェックで利用。

```

Cerebras llama3.1-8b (20s)
├─ 成功 → { score, summary, duplicateSuspected, usedModel } を返す
└─ 失敗
    └─ Groq llama-3.3-70b-versatile (15s)
        ├─ 成功 → 返す
        └─ 失敗 (429 TPD 超過 / タイムアウト等)
            └─ Gemini gemini-2.5-flash (30s)
                ├─ 成功 → 返す
                └─ 失敗 → エラー (UI でリトライ可能)

```

- マッチング理由は **150 字以内** (コミット **0d1af7e** で 100 字 → 150 字)
- 居住地・希望勤務地・案件備考・本人希望のスコア反映ロジックは **match-batch** の **calcRuleScore** 側に集約されているため、**match-score** は AI に丸投げで観点を指示する

auto-match (毎朝 JST 9:00 cron・コミット **aa480b8** で全面書き直し)

- app_config.auto_match_enabled='false'** でスキップ (既定 true)
- 直近 25 時間以内に登録された **prod** の案件を対象
- 既存ペア / **accepted** 状態の人材を除外
- JS 側スキル重複フィルタ (jsonb skills に && が使えないため includes でゆるい一致)
- 案件 1 件あたり最大 40 名 (**BATCH_AI_SIZE=20** × 2 リクエスト) を **match-batch** に渡す

- 失敗時は `errors[]` に集積して継続。`submissions upsert (onConflict: 'candidate_id,project_id', ai_raw: { autoMatched: true, source: 'auto-match-cron' })`

poll-email メール種別分類（任意・既定 OFF）

- `app_config.email_classify_enabled='true'` のときのみ動作
- 同一受信箱に `candidate / project` が混在するケース用
- Gemini `gemini-2.5-flash-lite` バッチで最大 20 件/コール、10 秒タイムアウト
- 失敗時は `candidate` にフォールバック
- ルールベース事前フィルタ (`SKIP_*_PATTERNS`, `PROJECT_*_PATTERNS`, `HR_SUBJECT_PATTERNS`) で自動判定できたメールは AI を呼ばずに分類

モデル一覧・特性

モデル	用途	無料枠	コンテキスト
Cerebras <code>llama3.1-8b</code>	<code>match-batch / match-score</code> の 1 段目	実質無制限	8K tokens
Groq <code>llama-3.3-70b-versatile</code>	<code>match-batch / match-score</code> の 2 段目	500K tokens/日（JST 9:00 リセット）	128K tokens
Groq <code>llama-3.1-8b-instant</code>	コード上は残存するが現状未使用	500K tokens/日	128K tokens
Gemini <code>gemini-2.5-flash</code>	<code>match-batch / match-score</code> 最終フォールバック	プリペイド制	1M tokens
Gemini <code>gemini-2.5-flash-lite</code>	<code>poll-email</code> 分類・ブラウザ補助	プリペイド制	1M tokens

Groq / Gemini 枯渇時の挙動（マッチング処理）

```
Cerebras → 失敗
├─ Groq → 429 Too Many Requests（日次上限超過）
│   └─ Gemini → 成功 → 正常動作
│       └─ 失敗（クレジットなし等）
│           ├── match-score: UI 側でエラー表示・リトライ可能
│           └── match-batch / auto-match: ルールスコアで全代替
(usedModel='rule')
```

JST 9:00 に Groq トークンがリセットされると自動復旧。3 段すべて失敗してもルールスコアでフォールバックされるため、`match-batch / auto-match` は止まらない。メール処理側は AI を使わないため、件数に関わらず無料で永続稼働する。

廃止済み（参考・歴史）

以下は 2026-05-19 のコミット [139a4f2](#) で廃止された旧フロー。コミット [a4dc3b4](#) で関数定義も **完全に削除**されており、現在のソースには残っていない。

旧 STEP1: 関連性チェック (`classifyInboundRelevance`)

```
Cerebras 8B (llama3.1-8b)
├─ 成功 → relevant: true/false
└─ 失敗
    └─ Groq 8B (llama-3.1-8b-instant)
        └─ 失敗
            └─ Gemini gemini-2.5-flash-lite
```

旧 STEP5: 人材情報抽出 (`generateJSONSmart kind='candidate'`)

```
画像添付あり OR (Cerebras/Groq キーなし)
└─ Gemini gemini-2.5-flash-lite (画像対応)

それ以外
└─ Groq llama-3.1-8b-instant
    └─ 失敗
        └─ Gemini gemini-2.5-flash-lite
```

- 廃止理由: Groq 無料枠 500K TPD = 約 125 件/日でメール取り込みが頻繁に詰まり、無限リトライループに陥っていた
- regex / 文章スキャン / `skill_master` 照合だけで実用精度を維持できると判断
- コミット [a4dc3b4](#) で `classifyInboundRelevance` / `generateJSONSmart` / `generateJSONWithCerebras` / `generateJSONWithGroq` / `generateJSON` (`kind='candidate'/'project'`) / `buildCandidateGroqPrompt` / `buildProjectGroqPrompt` を全て削除済み

最終更新: 2026-05-23